Spike Detection in MoStream

Porting di un'applicazione di DSPBench, verifica di correttezza, branch cooperative-runtime-v2 e campagna di misure — relazione tecnica, 29 ottobre 2026

Sintesi.

- È stata portata in MoStream l'applicazione **Spike Detection** di DSPBench (Source → MovingAverage → SpikeDetector → Sink), seguendo riga per riga la versione Storm, in una cartella nuova (`Benchmarks/SpikeDetection/`) senza modificare la libreria.
- Lo stato per sensore di MovingAverage è gestito con la **prima soluzione proposta dal professore**: un'unica coda fra gli stadi e un lock per ogni sensore. Il programma misura anche il prezzo di questa scelta: letture elaborate fuori ordine e attese sui lock.
- **Correttezza:** con MovingAverage a una replica, 30 esecuzioni su 30 (26 configurazioni: 2 code × 5 runtime × varie repliche) producono esattamente gli stessi spike dell'implementazione di riferimento. Con MovingAverage replicato si osservano letture fuori ordine, ma nelle 18 esecuzioni provate l'insieme degli spike non è cambiato.
- Il branch **cooperative-runtime-v2** è stato analizzato e reso utilizzabile con entrambe le code (Vyukov e Rigtorp) tramite lo script `prepare_v2.sh`. È emerso un **problema nel v2**: in modalità `coop-mpmc` il programma a volte si interrompe con "cooperative ready queue is full!". La causa individuata è una `try_push` che fallisce per contesa e viene trattata come errore fatale.
- **Campagna di misure** (400 esecuzioni, sezione 11): nel runtime standard **Rigtorp è più veloce di Vyukov del 24–49%** da 4+4 repliche in su, con un vantaggio che cresce con la contesa; con 1+4 sono alla pari. Nei runtime cooperativi le code sono alla pari. Il v2 è il cooperativo più veloce solo con `max_consume_rounds = 8` (fino a 1,9 volte); con il valore predefinito 1 è più lento del cooperativo della nostra versione.

1. Da dove partiamo

La richiesta del professore:

> *Se si riuscisse a scrivere in MoStream 1 o 2 applicazioni nuove avremmo un banco di prova più completo per testare le code e le nuove funzionalità che vogliamo introdurre, oltre a confrontare runtime standard vs cooperativo (anche quello nel branch cooperative-runtime-v2 che è più efficiente perché decentralizza le code pronti dei thread). […] una possibilità è quella di capire se si riesce a fare qualche applicazione in DSPBench.*

Spiegato semplice: cos'è MoStream

Immagina una fabbrica di biscotti fatta a catena di montaggio. Un operaio prende l'impasto, il secondo lo taglia, il terzo lo cuoce, l'ultimo lo mette nella scatola. Fra un operaio e l'altro c'è un **nastro trasportatore**: chi ha finito ci appoggia il pezzo, il successivo lo prende. MoStream serve a costruire fabbriche così, ma dentro il computer: gli "operai" si chiamano **stadi**, i "biscotti" sono i **dati** (i messaggi), e i "nastri" sono le **code**. Se un operaio è lento, se ne possono mettere più d'uno a fare lo stesso lavoro: si chiamano **repliche**.

1.1 Il lavoro precedente, in breve

Nelle due relazioni precedenti (`docs/Relazione_ImagePipeline_Rigtorp/`) abbiamo confrontato due code, Vyukov (quella di serie) e Rigtorp, dentro l'applicazione ImagePipeline. Il risultato: con immagini grandi le due code sono equivalenti, perché il tempo è tutto speso nei calcoli sulle immagini. Con immagini piccole e molti stadi replicati, nel runtime standard **Rigtorp è più veloce fino al 31%** e molto più stabile. Da qui l'interesse per un'applicazione in cui ogni messaggio richiede pochissimo lavoro, dove le code contano davvero: la Spike Detection è esattamente così.

> **Spiegato semplice: perché serviva un'altra applicazione**
> Se ogni operaio impiega un'ora per decorare un biscotto, non importa quanto sia veloce il nastro: il nastro aspetta sempre gli operai. Per capire quale nastro è migliore servono biscotti che si fanno in un secondo, così il nastro diventa importante. Le immagini erano "biscotti da un'ora"; la Spike Detection è fatta di "biscotti da un secondo".

2. DSPBench e l'applicazione Spike Detection

DSPBench (GMAP) raccoglie 16 applicazioni di stream processing, implementate per Storm, Flink e Spark. Il repository è stato scaricato in sola lettura e la Spike Detection è stata studiata sul codice della versione **Storm**, che è la più completa:

```
dspbench-storm/src/main/java/org/dspbench/applications/spikedetection/
  SpikeDetectionTopology.java   topologia (chi è collegato a chi e come)
  SensorParser.java             lettura delle righe del dataset
  MovingAverageBolt.java        media mobile per sensore
  SpikeDetectionBolt.java       rilevazione degli spike
dspbench-storm/src/main/resources/config/spikedetection.properties   parametri
```

2.1 La topologia

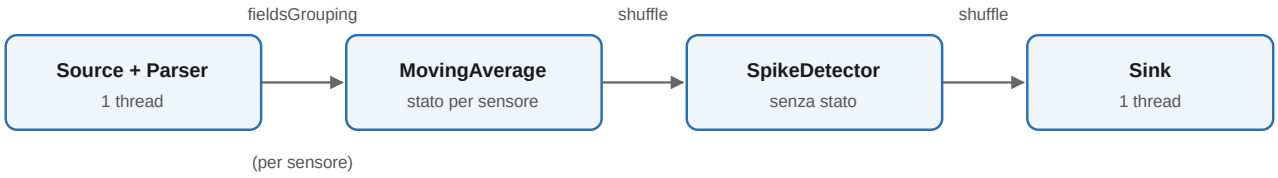


Figura 1 — Topologia della Spike Detection in DSPBench (Storm).

- **Source + Parser** (`SensorParser`): legge una riga del dataset, la divide sugli spazi e la accetta solo se ha esattamente 8 campi (`data ora epoch moteid temperatura umidità luce tensione`) e se data e ora sono valide. Emette `(deviceID, timestamp, valore)`; il valore è la temperatura (`sd.parser.value_field=temp`).
- **MovingAverage** (`MovingAverageBolt`): per ogni sensore tiene una lista degli ultimi valori e la loro somma. Se la lista ha già `window = 1000` valori toglie il più vecchio e lo sottrae dalla somma, poi aggiunge il nuovo e lo somma. Emette `(deviceID, media, valore)` con `media = somma / numero di valori`.
- **SpikeDetector** (`SpikeDetectionBolt`): emette `"spike detected"` solo se `|valore − media| > 0,03 × media` (`sd.spike_detector.threshold=0.03`).
- **Sink**: riceve gli spike (in DSPBench li stampa o li conta).
- Il collegamento Source→MovingAverage è un **`fieldsGrouping` per deviceID**: tutte le letture di uno stesso sensore vanno sempre alla stessa replica. Gli altri collegamenti sono

`shuffleGrouping`: ogni messaggio può andare a qualsiasi replica.

> **Spiegato semplice: cosa fa la Spike Detection**
> In una casa ci sono 52 termometri che ogni tanto mandano la temperatura. Per ogni termometro teniamo un quaderno con le ultime 1000 temperature e ne calcoliamo la media. Quando arriva una nuova temperatura la confrontiamo con la media: se è "troppo diversa" (più del 3% sopra o sotto), gridiamo "**salto!**", cioè uno *spike*. Per esempio, se la media è 20 gradi, il 3% è 0,6 gradi: una lettura di 21 gradi è uno spike, una di 20,3 no.

2.2 Il dataset

Il file `dspbench-spark/data/spikedetection/sensors.dat` (5 MB) contiene 80 000 righe del dataset del laboratorio Intel Berkeley. Dopo il parsing con le regole di DSPBench:

Voce	Valore
righe totali	80 000
righe scartate (numero di campi ≠ 8)	2 422
letture valide	**77 578**
sensori distinti	52 (ID da 1 a 54; mancano 5 e 28)
letture per sensore	da 496 (sensore 15) a 2 283 (sensore 48)
spike su una passata (riferimento)	59 862 (77% delle letture)

La quota di spike è alta: con una soglia del 3% sulla temperatura, la maggior parte delle letture si discosta dalla media delle ultime 1000. Di conseguenza il sink riceve circa tre quarti delle tuple: anche l'ultimo collegamento è molto trafficato.

3. Il problema della chiave in MoStream

In MoStream ogni collegamento fra due stadi è **una sola coda MPMC condivisa** da tutte le repliche (verificato in `MoStream/communicator.mojo` e `MoStream/stage.mojo`): qualsiasi replica può prendere qualsiasi messaggio. MoStream offre quindi solo l'equivalente dello `shuffleGrouping`, non del `fieldsGrouping`.

Se MovingAverage ha più repliche, le letture dello stesso sensore finiscono su repliche diverse. Se ogni replica avesse il suo stato privato, ciascuna calcolerebbe la media di una parte sola delle letture: il risultato sarebbe sbagliato e diverso a ogni esecuzione.

> **Spiegato semplice: il problema dei quaderni**
> Ora abbiamo quattro bambini che aiutano a calcolare le medie, e un solo nastro da cui ognuno prende la prossima lettura che passa. Se ogni bambino avesse il *suo* quaderno, le temperature del termometro della cucina finirebbero un po' nel quaderno di Anna, un po' in quello di Marco… e nessuno avrebbe la media giusta della cucina. In DSPBench il problema non c'è, perché c'è una regola: "le letture della cucina vanno sempre ad Anna". MoStream questa regola, per ora, non ce l'ha.

3.1 Bilanciamento del carico per chiave

Assegnando i sensori alle repliche con `id % n`, come farebbe un instradamento per chiave, il carico non è uniforme:

Repliche	Replica più carica / media
4	1,08
8	1,09
16	1,27

Con 16 repliche la più carica riceve il 27% di lavoro in più della media. È un costo dell'instradamento per chiave che la coda condivisa non ha.

4. Le due soluzioni del professore

> *Si mantiene la logica attuale ma i nodi consumatori tengono una lock per stato di chiave, e questo garantisce che lo stato è comunque acceduto in modo mutuamente esclusivo. Però non è la stessa cosa esattamente: se la sorgente è sequenziale l'implementazione originale garantisce che i dati della stessa chiave siano processati in ordine, cosa che noi non potremmo garantire con le lock. Comunque potremmo accontentarci.*

> *[…] se nella fase finale della tesi tu riuscissi ad impostare un Comunicatore diverso, potremmo provare e confrontare una soluzione come l'originale ma in MoStream. Ovvero fare in modo che se il Comunicatore è configurato opportunamente, consenta di realizzare primitive di `push_to` e `try_push_to` che consentano di mandare un dato ad uno specifico nodo nello stadio consumatore. […] A quel punto potremmo fare i confronti tra questa soluzione con il Comunicatore più complesso, e quella sopra con il comunicatore più semplice (singola coda) ma con lock.*

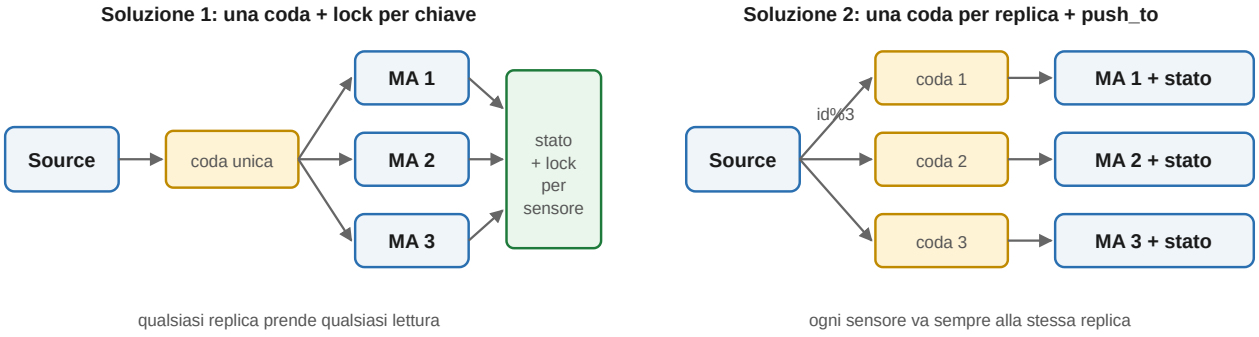


Figura 2 — Le due soluzioni proposte. In questa fase è stata realizzata la soluzione 1.

> **Spiegato semplice: le due soluzioni**
> **Soluzione 1:** c'è un solo nastro e i bambini prendono quello che passa, ma i quaderni sono uno per termometro, tutti su un tavolo comune, e ogni quaderno ha un **lucchetto**. Chi vuole scrivere nel quaderno della cucina prende il lucchetto; se ce l'ha già un altro aspetta. Così due bambini non scrivono mai nello stesso quaderno insieme. Il difetto: se Anna prende la lettura delle 10:00 della cucina e Marco quella delle 10:01, può succedere che Marco apra il lucchetto per primo e scriva la 10:01 prima della 10:00. L'ordine si è scambiato.
> **Soluzione 2:** ogni bambino ha il suo nastro e il suo quaderno, e chi mette le letture sui nastri segue la regola "cucina sempre ad Anna". Niente lucchetti e l'ordine è giusto, ma serve un sistema di nastri più complicato, e se la cucina manda tante letture Anna lavora più degli altri.

4.1 Considerazioni sulla soluzione 1

- `parallel(stage, n)` crea *n* copie dello stadio, quindi lo stato deve stare in memoria **condivisa**, raggiungibile tramite un puntatore che tutte le copie possiedono (come già fa `ImageSink` con il suo contatore).
- Il lock è uno **spinlock** su un intero atomico: la sezione critica è di pochi nanosecondi (una somma, una sottrazione e una scrittura), quindi aspettare "girando" costa meno che sospendere il thread.
- Nel runtime cooperativo uno spinlock è sicuro solo se l'actor non può cedere il controllo mentre lo tiene. `compute()` non contiene punti di cessione, quindi il lock viene sempre rilasciato prima che il worker passi a un altro actor.
- L'ordine per sensore non è garantito con più repliche: il programma conta quante letture arrivano "in ritardo", così il "potremmo accontentarci" diventa un dato misurato.

4.2 Considerazioni sulla soluzione 2 (per la fase successiva)

- Con una coda per replica consumatrice, ogni coda ha **un solo consumatore** (MPSC, o SPSC se il produttore è uno). Il `pop` bloccante di Rigtorp, che oggi il communicator non può usare, diventerebbe utilizzabile, e si potrebbero provare code specializzate per un solo consumatore.
- Va progettato l'invio dell'EOS su ogni coda e l'API con cui lo stadio produttore sceglie la destinazione. Per lo schema serve il documento iniziale del professore.

5. Il branch cooperative-runtime-v2

Il branch è stato scaricato da `https://github.com/ParaGroup/MoStream/tree/cooperative-runtime-v2` (ultimo commit `423b32b`, 9 ottobre 2026, "Fixed to compile with mojo 1.1.0"). Nel repository esistono anche `cooperative-runtime-v3` e `v4`; come indicato dal professore si usa il v2.

5.1 Cosa cambia

- **Code dei pronti per worker** (`ready_queue.mojo`, `work_stealing_deque.mojo`): invece di un'unica coda degli actor pronti condivisa da tutti i worker, ogni worker ha la sua.
- `run_cooperative(n_workers, ready_queue_kind = ReadyQueueKind.MPMC, max_consume_rounds = 1)`:
  - `ReadyQueueKind.MPMC`: una coda MPMC per worker (è il **valore predefinito**);
  - `ReadyQueueKind.WORK_STEALING`: una deque per worker, e un worker senza lavoro "ruba" actor agli altri;

    - `max_consume_rounds`: quante volte di fila un actor viene eseguito prima di essere rimesso in coda.
- Usa il modulo Python `multiprocessing` per contare le CPU, quindi a runtime serve Python (disponibile sulla VM: 3.10.12).

> **Spiegato semplice: runtime standard, cooperativo e v2**
> **Standard:** ogni bambino ha la sua postazione e ci resta tutto il tempo, anche quando non ha niente da fare.
> **Cooperativo:** le postazioni sono tante ma i bambini sono pochi. Una lavagna in mezzo alla stanza elenca le postazioni che hanno lavoro pronto: ogni bambino guarda la lavagna, va a una postazione, lavora un po' e poi torna alla lavagna. Se i bambini sono tanti, si accalcano tutti davanti all'unica lavagna.
> **v2:** ogni bambino ha la **sua** lavagnetta, così non si accalcano. Con il *work stealing*, chi ha la lavagnetta vuota sbircia quella del vicino e gli "ruba" un lavoro. Con *max_consume_rounds* = 8, un bambino arrivato a una postazione fa fino a 8 lavori di fila prima di tornare alla lavagnetta, invece di uno solo: perde meno tempo a camminare.

5.2 Compatibilità con il nostro lavoro

File	Rispetto alla nostra versione
stage.mojo, emitter.mojo, node.mojo, utils.mojo	identici: l'API degli stadi non cambia
MPMC_queue.mojo (Vyukov)	identico a meno dei commenti
communicator.mojo	differisce solo per la scelta della coda (4 righe)

Per usare Rigtorp nel v2 è stato scritto `prepare_v2.sh`: scarica il branch al commit `423b32b`, copia `communication_queue.mojo` e `MPMC_queue_rigtorp.mojo`, sostituisce le 4 righe del communicator (con controllo che il testo atteso sia presente esattamente una volta) e compila `libpinning.so`. Il risultato:

```
$ Benchmarks/SpikeDetection/prepare_v2.sh ~/mostream-v2
 MoStream/communicator.mojo | 8 ++++----
 1 file changed, 4 insertions(+), 4 deletions(-)
Ready: /home/ungolo/mostream-v2 (commit 423b32b, export MOSTREAM_HOME=/home/ungolo/mostream-v
2 before running)
```

Il repository del professore non viene modificato: la copia è in una cartella separata sulla VM.

6. Progetto dell'implementazione

6.1 Scelte generali

- **Il parsing avviene una volta, fuori dalla misura.** `prepare_dataset.py` applica le regole di `SensorParser` e scrive un file binario di coppie `(int64 sensore, float64 valore)`. Così: (a) Mojo e il riferimento Python leggono *gli stessi identici numeri*, e il confronto riguarda solo l'algoritmo; (b) il tempo misurato è quello della pipeline, non del parsing del testo. È una differenza dichiarata rispetto a DSPBench, dove il parsing avviene nella sorgente.
- **La sorgente riproduce il dataset in memoria**: per una durata fissa (modalità misura) oppure per un numero esatto di passate (modalità verifica). 77 578 letture si elaborano in pochi millisecondi, quindi per misurare serve ripeterle.
- **Ogni lettura riceve un numero di sequenza per sensore** (0, 1, 2, … per ciascun sensore), assegnato dalla sorgente. Serve per riconoscere gli spike (coppia sensore–

sequenza) e per contare le letture fuori ordine.
- **Il sink calcola un'impronta del risultato** indipendente dall'ordine di arrivo: somma modulo 2^{64} di un hash (splitmix64) di ogni coppia (sensore, sequenza). Due esecuzioni con lo stesso insieme di spike hanno la stessa impronta.

> **Spiegato semplice: l'impronta del risultato**
> Ogni spike riceve un "numero magico" calcolato dal suo termometro e dal suo numero d'ordine. Alla fine sommiamo tutti i numeri magici. Se due fabbriche hanno trovato esattamente gli stessi spike, la somma è identica, anche se li hanno trovati in un ordine diverso. Se anche uno solo è diverso, la somma cambia. È come un'impronta digitale del risultato.

6.2 I file

File	Contenuto
prepare_dataset.py	da `sensors.dat` al binario, con le regole di SensorParser
reference.py	implementazione sequenziale di riferimento: conteggio, impronta, elenco degli spike
compare_dump.py	confronta l'elenco degli spike di MoStream con quello del riferimento
sd_stages.mojo	messaggi, stato condiviso con lock per sensore, i quattro stadi
spike_detection.mojo	programma per la nostra versione di MoStream (standard e cooperativo)
spike_detection_v2.mojo	programma per il v2 (standard, coop-mpmc, coop-ws, rounds)
prepare_v2.sh	crea la copia del v2 con la scelta della coda
README.md	istruzioni complete

6.3 I messaggi

```
SensorReading  (device, seq, value)            Source → MovingAverage
AverageReading (device, seq, value, average)   MovingAverage → SpikeDetector
SpikeEvent     (device, seq, value, average)   SpikeDetector → Sink
```

Sono struct piccole di interi e numeri in virgola mobile: il costo di copia è minimo e il peso dell'applicazione è tutto nella comunicazione.

6.4 Lo stato condiviso e il lock

Lo stato di ogni sensore è una `DeviceState`: il lock, il numero di valori nella finestra, la posizione del più vecchio, la somma, la sequenza più alta vista e il puntatore alla finestra circolare di 1000 valori. La struct è **riempita fino a 64 byte** (una linea di cache), così i lock di sensori diversi non condividono la stessa linea (*false sharing*).

```
# spinlock acquire, returns the number of failed attempts
@always_inline
def acquire(mut self) -> Int:
    var waits = 0
    var expected = Int64(0)
    while not self.lock.compare_exchange[success_ordering=Ordering.ACQUIRE, failure_ordering=
Ordering.RELAXED](expected, Int64(1)):
        expected = Int64(0)
        waits += 1
    return waits

# spinlock release
@always_inline
def release(mut self):
    self.lock.store[ordering=Ordering.RELEASE](Int64(0))
```

L'acquisizione con ordine `ACQUIRE` e il rilascio con `RELEASE` garantiscono che chi prende il lock veda tutte le modifiche fatte da chi lo ha rilasciato.

> **Spiegato semplice: lo spinlock**
> Il lucchetto del quaderno è una porta con una maniglia. Chi vuole entrare prova la maniglia: se la porta si apre entra e la chiude dietro di sé; se è chiusa, riprova subito, e poi ancora, finché non si apre. Funziona bene quando chi è dentro esce in un attimo, come qui, dove si scrive solo un numero. Ogni tentativo andato a vuoto lo contiamo: sono le "attese sul lock".

6.5 MovingAverage

Esegue le stesse operazioni, nello stesso ordine, di `MovingAverageBolt`: prima sottrae il valore più vecchio, poi somma il nuovo. In virgola mobile l'ordine delle operazioni cambia il risultato nelle ultime cifre, e questo è necessario per avere esattamente gli stessi spike del riferimento.

```
def compute(mut self, var input: SensorReading) -> Optional[AverageReading]:
    var state = self.shared[].states.unsafe_offset(input.device)
    self.lock_waits += state[].acquire()
    if input.seq < state[].max_seq:
        self.late += 1
    else:
        state[].max_seq = input.seq
    if state[].size > WINDOW - 1:
        var oldest = state[].window.unsafe_offset(state[].head)
        state[].sum -= oldest[]
        oldest[] = input.value
        state[].head = (state[].head + 1) % WINDOW
    else:
        state[].window.unsafe_offset((state[].head + state[].size) % WINDOW)[] = input.value
        state[].size += 1
    state[].sum += input.value
    var average = state[].sum / Float64(state[].size)
    state[].release()
    self.count += 1
    return AverageReading(input.device, input.seq, input.value, average)
```

Una lettura è "in ritardo" (`late`) se la sua sequenza è minore della più alta già elaborata per quel sensore. I contatori `late` e `lock_waits` sono locali a ogni replica e vengono sommati in contatori atomici condivisi alla ricezione dell'EOS.

6.6 SensorSource, SpikeDetector e SpikeSink

- **SensorSource** legge il tempo solo ogni 1024 letture, per non spendere tempo nell'orologio; alla fine pubblica quante letture ha emesso.
- **SpikeDetector** è senza stato: applica la soglia e restituisce `None` se la lettura non è uno spike.
- **SpikeSink** conta gli spike, somma le impronte e, se richiesto, scrive l'elenco `sensore sequenza` in un file per il confronto puntuale.

6.7 L'uscita del programma

```
RESULT runtime=std queue=rigtorp ma=4 sd=4 workers=10 tuples=… averaged=… detected=… spikes=… hash=… late=… lock_waits=… elapsed_ms=… throughput=…
```

`tuples`, `averaged` e `detected` sono le tuple emesse dalla sorgente, elaborate da MovingAverage e da SpikeDetector: devono coincidere, altrimenti si è perso qualcosa. Il throughput è in tuple emesse dalla sorgente al secondo.

7. Errori trovati e corretti durante lo sviluppo

7.1 Data race sul contatore delle attese

Nella prima stesura il contatore delle attese sul lock stava dentro lo stato del sensore e veniva incrementato *mentre si aspettava* il lock. Più thread in attesa l'avrebbero scritto contemporaneamente senza protezione: una *data race*, quindi un conteggio non affidabile. Il contatore è stato spostato in una variabile locale di ogni replica, pubblicata con un atomico alla fine.

> **Spiegato semplice**
> Avevamo appeso alla porta un foglietto su cui chi aspetta fa una tacca. Ma se tre bambini aspettano insieme e fanno la tacca nello stesso momento, qualche tacca si perde. Ora ognuno conta le sue attese sul proprio dito e alla fine le dice ad alta voce.

7.2 Frazioni di secondo a una o due cifre

Il formato data di DSPBench è `appendFractionOfSecond(3, 6)` di Joda-Time. All'inizio era stato interpretato come "da 3 a 6 cifre" e 12 righe con frazione a 2 cifre (ad esempio `2004-02-28 01:15:16.09`) venivano scartate. In Joda-Time quei parametri indicano il numero di cifre da *stampare*; in lettura si accettano da 1 a 6 cifre, quindi DSPBench accetta quelle righe. Dopo la correzione le letture valide sono 77 578, con 0 righe scartate per la data.

8. Verifica di correttezza

8.1 Prove preliminari

Prima di scrivere il codice è stato verificato su Mojo 1.1 che un `float64` scritto da Python con `struct.pack("<d", 19.7336)` e riletto in Mojo con `read_bytes` e `bitcast[DType.float64]` sia identico al valore originale (`19.7336 True`).

8.2 Riferimento Python

```
$ python3 reference.py data/sensors.bin 1
RESULT tuples=77578 spikes=59862 hash=5331604604058402807
$ python3 reference.py data/sensors.bin 3
RESULT tuples=232734 spikes=200774 hash=6159549295451861730
```

8.3 MovingAverage a una replica: risultato esatto

Libreria	Code	Runtime	Altre variabili	Esecuzioni	Esito
nostra versione	Vyukov, Rigtorp	std, coop	passate 1 e 3; SpikeDetector 1 e 4	16	**tutte identiche**
nostra versione	Vyukov, Rigtorp	std, coop	3 passate, SD=4, con elenco spike	4	**tutte identiche**
v2	Vyukov, Rigtorp	std, coop-mpmc, coop-ws	3 passate, SD=4, rounds 1 e 8	10	**tutte identiche**

"Identiche" significa: stesse tuple emesse, elaborate e controllate, stesso numero di spike, stessa impronta, nessuna lettura fuori ordine e, dove è stato scritto l'elenco, stesso insieme di coppie (sensore, sequenza) del riferimento.

8.4 MovingAverage replicato

Libreria	Coda	Runtime	MA	Letture in ritardo	Attese sul lock	Insieme degli spike
nostra	Vyukov	std	4	17	127	**identico**
nostra	Vyukov	std	16	0	31	**identico**
nostra	Vyukov	coop	4	1	0	**identico**
nostra	Vyukov	coop	16	0	0	**identico**
nostra	Rigtorp	std	4	10	102	**identico**
nostra	Rigtorp	std	16	4	454	**identico**
nostra	Rigtorp	coop	4	1	0	**identico**
nostra	Rigtorp	coop	16	0	0	**identico**
v2	Vyukov	std / coop-mpmc r1, r8 / coop-ws r1, r8	4	12 / 1, 12 / 1, 16	129 / 5, 41 / 0, 21	**identico**
v2	Rigtorp	std / coop-mpmc r1, r8 / coop-ws r1, r8	4	3 / 0, 29 / 3, 19	107 / 0, 55 / 12, 42	**identico**

3 passate (232 734 tuple), SpikeDetector a 4 repliche.

Le letture fuori ordine **si verificano davvero**, come previsto dal professore. Nelle 18 esecuzioni provate però nessuno scambio ha cambiato una decisione spike/non spike: scambiare due letture vicine sposta la media di pochissimo rispetto alla soglia del 3%. Le medie emesse invece possono differire da quelle del riferimento. Con esecuzioni più lunghe e più repliche le letture in ritardo crescono molto (sezione 9), quindi non si può concludere che l'insieme degli spike resti sempre identico: andrà misurato nella campagna.

Spiegato semplice: come sappiamo che funziona
Abbiamo scritto due volte lo stesso programma: una volta "lento e semplice", con un solo bambino che fa tutto in ordine (il riferimento), e una volta con la fabbrica MoStream. Poi abbiamo controllato che trovino esattamente gli stessi spike, con la stessa impronta digitale, in 30 prove diverse su 30. Quando i bambini che calcolano le medie sono più di uno, ogni tanto due letture si scambiano di posto: le abbiamo contate, e finora non hanno cambiato nessuno spike.

9. Prime misure (indicative)

Una sola esecuzione da 10 secondi per caso, senza riscaldamento né alternanza: servono solo a orientare la campagna, non a trarre conclusioni. I risultati della campagna sono nella sezione 11. Il numero di worker cooperativi è pari al numero di actor (MA + SD + 2).

9.1 Nostra versione di MoStream

MA + SD	Std Vyukov	Std Rigtorp	Coop Vyukov	Coop Rigtorp	Late / attese std V	Late / attese std R
1 + 1	1,52 M	2,35 M	0,51 M	0,51 M	0 / 0	0 / 0
1 + 4	2,24 M	2,18 M	0,39 M	0,39 M	0 / 0	0 / 0
4 + 4	3,51 M	4,39 M	0,38 M	0,39 M	1 904 / 21 947	1 880 / 38 895
8 + 8	2,72 M	3,90 M	0,30 M	0,31 M	325 / 4 046	1 079 / 15 800
16 + 16	1,72 M	2,68 M	0,27 M	0,26 M	204 / 4 217	414 / 8 354

Throughput in tuple al secondo.

9.2 Branch v2

MA + SD	Coda	std	coop-mpmc r1	coop-mpmc r8	coop-ws r1	coop-ws r8
1 + 4	Vyukov	1,30 M	0,29 M	*errore*	0,32 M	0,66 M
1 + 4	Rigtorp	2,06 M	*errore*	0,61 M	0,31 M	0,68 M
4 + 4	Vyukov	3,02 M	0,32 M	0,74 M	0,36 M	0,78 M
4 + 4	Rigtorp	4,08 M	0,27 M	0,66 M	0,33 M	0,75 M
16 + 16	Vyukov	1,49 M	0,11 M	0,21 M	0,12 M	0,26 M
16 + 16	Rigtorp	2,70 M	0,11 M	0,23 M	0,13 M	0,26 M

"errore": l'esecuzione si è interrotta con "cooperative ready queue is full!" (sezione 10).

9.3 Prime indicazioni

- **Runtime standard:** Rigtorp avanti in quasi tutte le configurazioni (da +25% a +56%), alla pari con 1+4. Il massimo, circa 4,4 M tuple/s, si raggiunge con 4+4; oltre, il throughput scende: più repliche significano più contesa sulle code e sui lock, e la sorgente resta una sola.
- **Cooperativo:** da 4 a 10 volte più lento dello standard, e peggiora aggiungendo repliche, con entrambe le code: il limite è lo scheduler.
- **v2:** `max_consume_rounds = 8` raddoppia il throughput del cooperativo; il work stealing aggiunge circa il 10–20%.
- **Da verificare:** con 16+16 e `rounds = 1` il v2 (0,11–0,13 M) sembra più lento del cooperativo della nostra versione (0,26 M); anche il runtime standard del v2 risulta più

basso in alcuni casi (1,30 M contro 2,24 M con Vyukov a 1+4), anche se il codice del runtime standard è lo stesso. Con una sola esecuzione non si può distinguere un effetto reale dal rumore: la campagna lo chiarirà.
- La soluzione 1 ha un costo visibile: con 4+4 nel runtime standard, in 10 secondi, circa 1 900 letture fuori ordine e fino a 39 000 attese sul lock.

10. Un problema nel v2: "cooperative ready queue is full!"

10.1 Il sintomo

Prova	Riuscite	Fallite
v2, Rigtorp, coop-mpmc, rounds 1, 1+4, 7 worker, 10 s	3	3
v2, Vyukov, coop-mpmc, rounds 1, 1+4, 7 worker, 3 s	6	2
v2, Vyukov, coop-ws, rounds 1, 1+4, 7 worker, 3 s	8	0
v2 originale, Vyukov, coop-mpmc, 2 s	17	3
v2 con `push_local` che ritenta, Vyukov, coop-mpmc, 2 s	20	0

Il problema si presenta anche con **Vyukov**, quindi non dipende dall'integrazione di Rigtorp.

10.2 La causa

Le code dei pronti `coop-mpmc` sono code Vyukov di capacità pari alla potenza di due successiva a `actor + 1`: 8 per 7 actor. Una coda davvero piena richiederebbe lo stesso actor inserito due volte. Il punto è un altro: la `try_push` di Vyukov in MoStream fa **un solo tentativo**.

```
def try_push(mut self, var item: Self.T) -> Optional[Self.T]:
    var pw = self.enqueue_pos.atomicVal.load[ordering=Ordering.RELAXED]()
    var cell_ptr = self.buffer.unsafe_offset(pw & self.mask)
    var seq = cell_ptr[].sequence.load[ordering=Ordering.ACQUIRE]()
    if pw != seq:
        return Optional(item^) # queue is currently full
    if not self.enqueue_pos.atomicVal.compare_exchange[...](pw, pw + 1):
        return Optional(item^) # queue is currently full
    ...
```

Se un altro produttore ha appena inserito un elemento, la CAS fallisce, oppure la sequenza letta non corrisponde più alla posizione, e la funzione restituisce l'elemento **come se la coda fosse piena**. Nell'algoritmo originale di Vyukov in questi casi si rilegge la posizione e si riprova; "piena" si dichiara solo quando la sequenza della cella è *minore* della posizione. Nel communicator questo è innocuo, perché chi chiama riprova più tardi. Nel v2 invece:

```
def push_local(mut self, worker_id: Int, actor_id: Int64) -> Bool:
    var rejected = self.queues.unsafe_offset(worker_id)[].try_push(actor_id)
    return not rejected
# scheduler.mojo
if not self.ready_queues.push_local(worker_id, Int64(actor.flat_id)):
    print_red_color("{MoStream} Error: cooperative ready queue is full!")
```

Quando due worker inseriscono un actor nella stessa coda dei pronti nello stesso momento, uno dei due fallisce per contesa e il programma si interrompe.

10.3 Conferma e possibile correzione

In una copia separata (`~/mostream-v2-retry`, solo per diagnosi) `push_local` ripete `try_push` finché riesce: 20 esecuzioni su 20 sono terminate correttamente. Le correzioni possibili, da valutare con il professore:

- rendere la `try_push` di Vyukov fedele all'algoritmo originale: riprovare quando la CAS fallisce o la posizione è cambiata, e dichiarare "piena" solo se la sequenza è minore della posizione. Migliorerebbe anche il communicator, che oggi può segnalare "piena" a torto;
- oppure, nel v2, ripetere il tentativo in `push_local`, visto che per costruzione la coda dei pronti ha spazio per tutti gli actor.

Il branch del professore non è stato modificato. Il modo `coop-ws` non ha mostrato il problema.

> **Spiegato semplice: il guasto del v2**
> Ogni bambino ha una cassettina dove si mettono i biglietti dei lavori pronti, e c'è sempre posto per tutti. Ma la cassettina ha una regola strana: se due persone provano a infilarci un biglietto *nello stesso istante*, una delle due viene respinta e pensa "la cassettina è piena!". Nel v2, quando qualcuno pensa che la cassettina sia piena, suona l'allarme e la fabbrica si ferma. La cassettina non era piena: bastava riprovare un attimo dopo. Infatti, quando abbiamo insegnato a riprovare, la fabbrica non si è più fermata.

11. Risultati della campagna

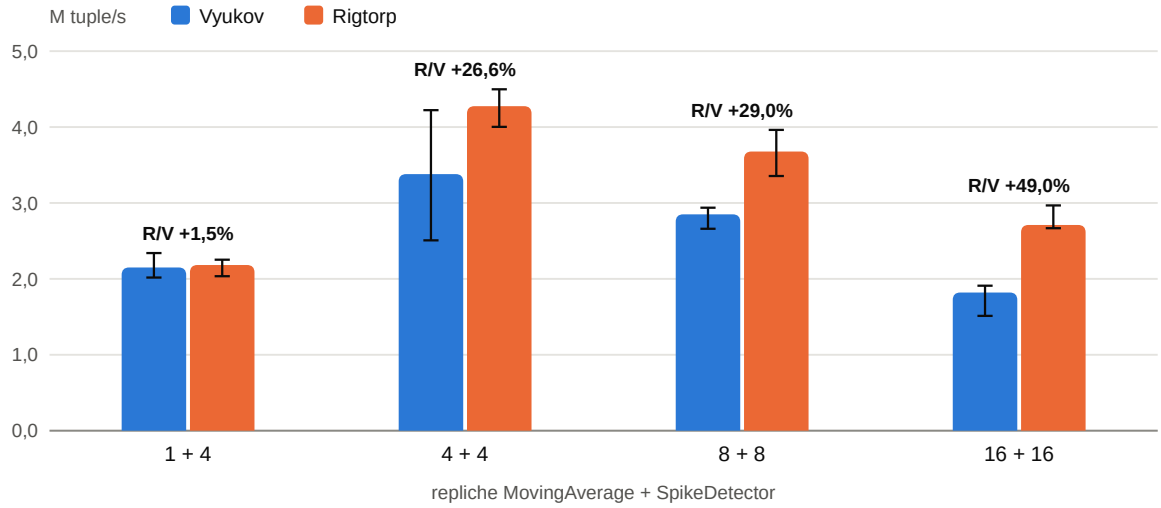
> **Protocollo.** 400 esecuzioni da 10 secondi sulla VM: 4 configurazioni (MA + SD = 1+4, 4+4, 8+8, 16+16) × 5 runtime × 2 code × 10 ripetizioni. In ogni ripetizione le due code girano una dopo l'altra, con ordine alternato (Vyukov prima nelle ripetizioni dispari, Rigtorp prima nelle pari); una esecuzione di riscaldamento iniziale è scartata. Worker cooperativi = numero di actor (MA + SD + 2). Script: `run_campaign.sh`; dati grezzi: `Benchmarks/SpikeDetection/risultati/campagna_20261029.csv` e `.log`.
> **Integrità:** 400 righe su 400 con risultato, nessun timeout, nessuna coda diversa da quella attesa, e in tutte `tuples = averaged = detected`: nessuna tupla persa.

11.1 Tabella completa

MA + SD	Runtime	Vyukov	CV V	Rigtorp	CV R	R / V	Coppie vinte da R	p (segno)
1 + 4	Nostra, standard	2,15	5,3%	2,18	3,2%	+1,5%	7/10	0,344
	v2, standard	2,11	4,6%	2,06	5,4%	−2,0%	5/10	1,000
	Nostra, cooperativo	0,40	1,5%	0,39	1,8%	**−3,3%**	1/10	0,021
	v2, coop-ws, rounds 1	0,34	3,7%	0,32	5,5%	−4,9%	2/10	0,109
	v2, coop-ws, rounds 8	0,63	3,4%	0,63	3,8%	−0,4%	5/10	1,000
4 + 4	Nostra, standard	3,38	17,9%	4,28	4,2%	+26,6%	8/10	0,109
	v2, standard	3,41	11,8%	4,24	3,8%	**+24,3%**	9/10	0,021
	Nostra, cooperativo	0,39	1,8%	0,38	2,2%	−0,8%	6/10	0,754
	v2, coop-ws, rounds 1	0,34	4,7%	0,33	7,5%	**−4,7%**	1/10	0,021
	v2, coop-ws, rounds 8	0,72	6,5%	0,74	7,3%	+1,8%	5/10	1,000
8 + 8	Nostra, standard	2,85	3,0%	3,68	5,0%	**+29,0%**	10/10	0,002
	v2, standard	2,85	1,9%	3,75	4,3%	**+31,6%**	10/10	0,002
	Nostra, cooperativo	0,31	1,0%	0,30	1,2%	−1,6%	2/10	0,109
	v2, coop-ws, rounds 1	0,20	7,4%	0,18	6,4%	**−10,1%**	1/10	0,021
	v2, coop-ws, rounds 8	0,43	5,6%	0,43	6,2%	+1,1%	6/10	0,754
16 + 16	Nostra, standard	1,82	6,7%	2,71	3,1%	**+49,0%**	10/10	0,002
	v2, standard	1,84	17,8%	2,69	0,8%	**+45,9%**	10/10	0,002
	Nostra, cooperativo	0,27	1,2%	0,27	0,8%	−1,0%	3/10	0,344
	v2, coop-ws, rounds 1	0,13	6,9%	0,13	8,0%	−0,7%	6/10	0,754
	v2, coop-ws, rounds 8	0,25	5,3%	0,26	6,1%	+1,2%	6/10	0,754

Throughput mediano in milioni di tuple/s su 10 ripetizioni. CV = deviazione standard / media. "Coppie vinte da R": in quante ripetizioni Rigtorp ha superato Vyukov nella stessa ripetizione. p = test del segno bilaterale sulle 10 coppie (con 10 coppie il minimo possibile è 0,002); in grassetto le differenze con $p < 0,05$.

11.2 Runtime standard: Rigtorp vince quando la contesa cresce



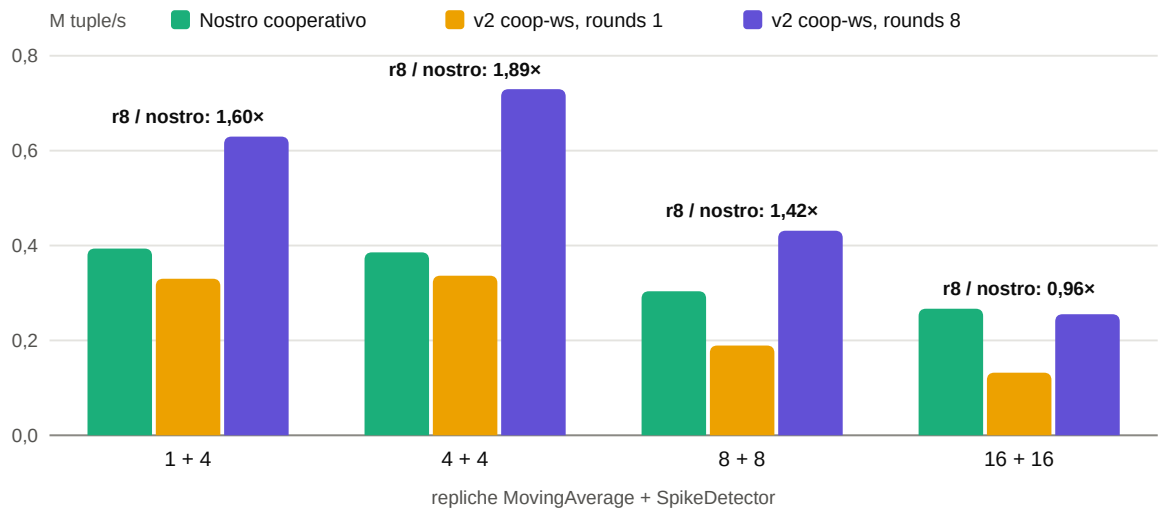
Runtime standard della nostra versione. Barre: mediana su 10 ripetizioni; linee: minimo–massimo. Il v2 standard dà valori quasi identici (tabella 11.1).

- Con **1+4** le due code sono alla pari (+1,5% e −2,0%, nessuna differenza significativa): una sola replica di MovingAverage legge dalla prima coda e la contesa è bassa.

- Con **4+4** Rigtorp è avanti del 24–27% sulle mediane. Nella nostra versione la differenza non supera il test del segno (8/10, p = 0,11) perché Vyukov è molto instabile (CV 17,9%, contro 4,2% di Rigtorp): in due ripetizioni Vyukov ha fatto meglio. Nel v2 è significativa (9/10).
- Con **8+8** e **16+16** Rigtorp vince 10 ripetizioni su 10 in entrambe le librerie, con un vantaggio che cresce: +29–32% e +46–49%.
- Il throughput massimo, circa 4,3 M tuple/s, si ha con 4+4 e Rigtorp. Oltre, scende per entrambe le code (più thread che contendono code e lock, un'unica sorgente), ma Vyukov scende di più: −46% da 4+4 a 16+16 contro −37% di Rigtorp.
- Nostra versione e v2 coincidono entro il 2–3% in tutte le configurazioni. Il valore di 1,30 M del v2 standard nelle prime misure (sezione 9.2) era rumore: il codice del runtime standard è lo stesso, e ora lo sono anche i numeri.

Il risultato conferma, su un'applicazione diversa, quanto visto con ImagePipeline: la `push` bloccante di Rigtorp prende il posto con un `fetch_add` che non fallisce mai, mentre con Vyukov i produttori ripetono CAS fallite tanto più spesso quanti più sono i thread sulla stessa coda.

11.3 Runtime cooperativi: le code contano poco, lo scheduler molto



Throughput dei runtime cooperativi: media delle mediane di Vyukov e Rigtorp, che differiscono di pochi punti percentuali (tabella 11.1). Sopra ogni gruppo: rapporto fra v2 con rounds 8 e il nostro cooperativo.

- **Code alla pari o con un lieve vantaggio di Vyukov.** Le differenze vanno da −10,1% a +1,8%; quelle significative sono tutte a favore di Vyukov (nostro coop 1+4, v2 rounds 1 a 4+4 e 8+8). Nel cooperativo un actor alla volta usa una coda e la contesa è bassa, quindi il vantaggio di Rigtorp non si manifesta, mentre la sua `try_pop` (un ciclo di CAS) costa un po' più di quella di Vyukov.
- **Il cooperativo è 5–10 volte più lento dello standard** con qualsiasi coda: il collo di bottiglia è lo scheduler.
- **Il v2 migliora solo con `max_consume_rounds = 8`:** 1,6 volte il nostro cooperativo con 1+4, 1,9 volte con 4+4, 1,4 volte con 8+8; con 16+16 il vantaggio sparisce (0,26 contro 0,27 M).
- **Con il valore predefinito `rounds = 1` il v2 è sempre più lento** del nostro cooperativo: −15% con 1+4, −37% con 8+8, circa la metà con 16+16. È il risultato più importante da riportare al professore, visto che il branch nasce per essere più efficiente.
- Un'ipotesi, *non verificata*, per il calo con molti actor: con molti worker cresce il costo di cercare lavoro da rubare, che supera il guadagno della decentralizzazione; inoltre con 34

worker su 28 core fisici entrano in gioco gli hyperthread. Per verificarla servirebbero misure con meno worker che actor e un profilo dei worker.

11.4 Il prezzo della soluzione 1: letture fuori ordine e attese sui lock

MA + SD	Runtime	Late V	Late R	Attese lock V	Attese lock R
4 + 4	Nostra, standard	1 250	1 536	20 182	26 535
	v2, standard	1 406	1 491	22 191	24 836
	Nostra, cooperativo	16	12	36	26
	v2, coop-ws, rounds 1	28	30	56	53
	v2, coop-ws, rounds 8	500	524	906	798
8 + 8	Nostra, standard	365	832	8 436	14 581
	v2, standard	368	804	8 300	12 389
	Nostra, cooperativo	13	12	14	13
	v2, coop-ws, rounds 1	14	7	22	13
	v2, coop-ws, rounds 8	346	348	332	306
16 + 16	Nostra, standard	195	386	1 739	9 664
	v2, standard	196	408	1 848	7 978
	Nostra, cooperativo	24	26	49	64
	v2, coop-ws, rounds 1	9	10	8	1
	v2, coop-ws, rounds 8	191	185	109	122

Mediane su 10 ripetizioni da 10 s. "Late": letture elaborate dopo una lettura successiva dello stesso sensore. "Attese lock": tentativi falliti di acquisire il lock di un sensore. Con 1+4 entrambi sono sempre 0 (una sola replica di MovingAverage).

- Nel runtime standard le letture fuori ordine sono da 200 a 1 500 per esecuzione, cioè da circa 1 ogni 28 000 tuple (4+4, Rigtorp) a 1 ogni 90 000 (16+16, Vyukov). Sono più frequenti con 4+4, dove il throughput è massimo.
- Nel runtime standard Rigtorp ha più attese sui lock di Vyukov: le repliche di MovingAverage ricevono più dati al secondo e si contendono i lock più spesso. Le attese sono comunque poche rispetto alle tuple (circa 1 ogni 1 500 con 4+4).
- Nel cooperativo con `rounds = 1` fuori ordine e attese sono quasi assenti (decine). Con `rounds = 8` salgono a 190–520: un actor consuma più messaggi di fila mentre un'altra replica, su un altro worker, lavora sugli stessi sensori.
- Questi numeri dicono *quanto spesso* l'ordine si rompe, non se cambia il risultato: la verifica sugli spike (sezione 8) è stata fatta su esecuzioni brevi. Misurarne l'effetto su esecuzioni lunghe è uno dei prossimi passi.

11.5 Conclusioni della campagna

1. Per il runtime standard **Rigtorp è la coda migliore** appena più thread condividono le code: +24–49%, più stabile, e il vantaggio cresce con le repliche. Con poca contesa le due code si equivalgono.
2. Per i runtime cooperativi **la scelta della coda è quasi indifferente**, con un piccolo vantaggio di Vyukov in alcuni casi.
3. Il **v2 conviene solo con `max_consume_rounds` maggiore di 1** e fino a un numero moderato di actor. Con il valore predefinito è più lento del cooperativo della nostra versione.

4. Il costo della soluzione 1 (lock per sensore) è misurabile ma piccolo in termini di attese; il punto aperto è l'effetto delle letture fuori ordine sul risultato.

> **Spiegato semplice: cosa abbiamo scoperto**
> Abbiamo fatto correre la fabbrica 400 volte, 10 secondi ogni volta, cambiando il numero di bambini e il tipo di nastro trasportatore. Quando tutti i bambini lavorano insieme (runtime standard) e sono tanti, il nastro Rigtorp è molto meglio: è come un distributore di numeretti, ognuno prende il suo e nessuno litiga. Con il nastro Vyukov invece i bambini si spingono per prendere il posto, e più sono più si spingono: con 32 bambini Rigtorp fa passare metà dei pacchi in più.
> Quando invece i bambini lavorano a turno (runtime cooperativo), non si spingono mai, quindi il tipo di nastro non conta: conta solo quanto è bravo il capo a decidere i turni. Il capo nuovo (v2) è più bravo solo se lascia lavorare ogni bambino un po' più a lungo prima di cambiare turno (rounds 8). Se cambia turno a ogni pacco (rounds 1), perde più tempo del capo vecchio.

12. Prossimi passi

12.1 La campagna di misure (svolta, sezione 11)

- **Code:** Vyukov e Rigtorp.
- **Runtime:** standard e cooperativo della nostra versione; v2 standard e `coop-ws` con `rounds` 1 e 8. `coop-mpmc` è escluso per il problema della sezione 10.
- **Configurazioni MA + SD:** 1+4, 4+4, 8+8, 16+16; worker cooperativi pari al numero di actor.
- **Protocollo:** esecuzioni da 10 s, ripetizioni con ordine Vyukov/Rigtorp alternato, un riscaldamento scartato, controllo di `tuples = averaged = detected` e della terminazione corretta, registrazione di `late` e `lock_waits`.

12.2 Dopo la campagna

- Riportare al professore il problema di `coop-mpmc` (sezione 10) e il calo del v2 con `rounds = 1` e con molti actor (sezione 11.3); chiedere se considerare anche i branch v3 e v4.
- Capire il calo del v2 con molti actor: misure con meno worker che actor e profilo dei worker.
- Misurare quanto le letture fuori ordine cambiano gli spike nelle esecuzioni lunghe con MovingAverage replicato.
- Progettare e realizzare il communicator a più code con `push_to`/`try_push_to` (soluzione 2), con il documento del professore, e confrontarlo con la soluzione 1.
- Scegliere la seconda applicazione con un profilo diverso, ad esempio Word Count (stadio uno-a-molti) o un'applicazione con più calcolo per tupla.

Appendice A — Comandi

Preparazione

```
# DSPBench, in sola lettura
git clone --depth 1 https://github.com/GMAP/DSPBench.git
# dataset binario e riferimento
python3 Benchmarks/SpikeDetection/prepare_dataset.py sensors.dat Benchmarks/SpikeDetection/da
ta/sensors.bin
# records=77578 devices=52 max_device=54 skipped={'fields': 2422, 'date': 0, 'key': 0, 'valu
e': 0}
python3 Benchmarks/SpikeDetection/reference.py Benchmarks/SpikeDetection/data/sensors.bin 3 r
ef_3.txt
```

Nostra versione di MoStream (VM, ~/mostream-rigtorp-bench)

```
export MOSTREAM_HOME="$PWD"
mojo build -O3 -I . -I Benchmarks/SpikeDetection Benchmarks/SpikeDetection/spike_detection.mo
jo -o bin/sd_vyukov
mojo build -O3 -I . -I Benchmarks/SpikeDetection -D MOSTREAM_RIGTORP=1 Benchmarks/SpikeDetect
ion/spike_detection.mojo -o bin/sd_rigtorp
# ./sd_<coda> <dataset.bin> <durata_s> <passate> <MA> <SD> <std|coop> <n_workers> [dump]
bin/sd_rigtorp Benchmarks/SpikeDetection/data/sensors.bin 0 3 1 4 std 6 d.txt     # verifica
python3 Benchmarks/SpikeDetection/compare_dump.py ref_3.txt d.txt
bin/sd_rigtorp Benchmarks/SpikeDetection/data/sensors.bin 10 0 4 4 std 10         # misura
```

Branch v2 (VM, ~/mostream-v2)

```
Benchmarks/SpikeDetection/prepare_v2.sh ~/mostream-v2
cd ~/mostream-v2 && export MOSTREAM_HOME="$PWD"
B=~/mostream-rigtorp-bench/Benchmarks/SpikeDetection
mojo build -O3 -I . -I $B $B/spike_detection_v2.mojo -o bin/sd_v2_vyukov
mojo build -O3 -I . -I $B -D MOSTREAM_RIGTORP=1 $B/spike_detection_v2.mojo -o bin/sd_v2_rigto
rp
# ./sd_v2_<coda> <dataset.bin> <durata_s> <passate> <MA> <SD> <std|coop-mpmc|coop-ws> <n_work
ers> <rounds> [dump]
bin/sd_v2_rigtorp $B/data/sensors.bin 10 0 4 4 coop-ws 10 8
```

Appendice B — Ambiente

Voce	Valore
Macchina	VM `ungolo@131.114.2.95`, Intel Xeon Gold 5512U, 28 core / 56 thread, 1 nodo NUMA
Compilatore	Mojo 1.1.0 (8189361e), opzione `-O3`
Python	3.10.12 sulla VM (preparazione, riferimento, v2)
Nostra libreria	sorgenti con gli stessi hash dei test di integrazione del 26 ottobre
v2	ParaGroup/MoStream, branch cooperative-runtime-v2, commit 423b32b, più la scelta della coda
DSPBench	GMAP/DSPBench, clone del 29 ottobre 2026 (ramo principale)
Capacità delle code fra stadi	1024 (predefinita)
Pinning	attivo